

SEGMENT 05 • HANDS-ON

Indus and the agentic platform

From APIs you assemble to agents you deploy

indus.sarvam.ai • six tools • work, voice, content, doc, coding agents + inference

Dr. Bhaveshkumar C. Dharmani • PhD (ICT), DA-IICT Gandhinagar • Founder, AI Vidhya4Sarvam

<https://www.aividhya.in/> • <https://www.aividhya4sarvam.in/> • bhavesh@aividhya.in • <https://www.linkedin.com/in/bhaveshdharmani/>

What actually makes something 'agentic'

1 A chatbot

- Answers a question
- Stateless, or state you manage yourself
- One turn in, one turn out
- Cannot act on the world
- Failure = a bad answer
- You evaluate it on response quality

2 An agent

- Completes a task
- Holds state across many steps, sometimes days
- Decides which tools to call, and in what order
- Takes actions — writes to a CRM, raises a ticket, moves money
- Failure = a wrong action, which may be irreversible
- You evaluate it on task success, tool accuracy and recovery

★ The moment an LLM can take an action, the engineering problem stops being prompt quality and becomes reliability, observability and blast radius.

Indus — one platform, six tools

W

Work agents

Multi-step business workflows. The general-purpose surface — compliance review, onboarding, internal ops.

V

Voice agents

Conversational systems over telephony and web. The Samvaad capability, exposed as a building block.

C

Content agents

Generation, transformation and localisation of content at scale across Indian languages.

D

Doc agents

Document ingestion, digitisation, extraction and downstream routing. Vision 1.5 with a workflow around it.

K

Coding agents

Sarvam Code — writing and understanding code. Beta.

I

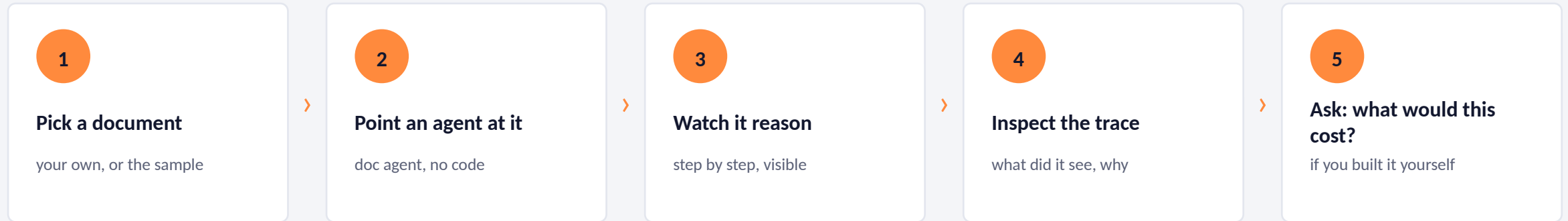
Inference stack

Sarvam Inference — India-hosted serving for 105B and frontier open models, underneath all of it.



Create your account at indus.sarvam.ai now if you have not. We are going to run a real task through it together in the next ten minutes.

What we are doing, together, right now



 **Narrate the layers underneath as it runs**

That is Vision doing extraction. That is 105B doing the reasoning. That is a tool call. Everything on this screen, you could assemble yourself from the APIs we ran in the last segment.

 **Which raises the actual question of the day**

You could build this. Sometimes you should. Sometimes you absolutely should not. We settle that with arithmetic in the next segment.

B

The anatomy of a production agent

The checklist an enterprise buyer will hold you to

Seven things that separate a demo from a product

1

Tools

Scoped, typed, individually permissioned. The agent should reach only what that task needs — not your whole database.

2

State

Where is the conversation, the task, the partial result? Held explicitly, not implicitly in a growing context window.

3

Memory

What persists across sessions? Structured feedback loops mean agents learn from corrections rather than repeating them.

4

Checkpointing

A failure at step 40 must not lose the first 39. This is the single biggest reliability difference at scale.

5

Recovery

Resume from the last good step, not from zero. Instant, automatic, and tested before you need it.

6

Observability

Every reasoning step, tool call and decision logged and inspectable. Replay any step with different inputs.

7

Audit trail

Who did what, when, and on what evidence. Not a nice-to-have in BFSI or government — a procurement requirement.



Arya is built around exactly this list. You are not learning their product — you are learning the questions a CISO will ask you in month three.

The design law worth writing on your wall



The expensive way

- Ask the model to compute the EMI
- Ask the model to sort 200 records
- Ask the model to validate a PAN format
- Ask the model to add up an invoice
- Non-deterministic, expensive, occasionally wrong
- Costs scale with every token of data you paste in



The right way

- Model decides WHICH calculation is needed
- Code performs it, deterministically
- Model interprets the result and decides what next
- Deterministic, auditable, free at the point of execution
- Arya reports 114% better on complex tasks — with smaller, cheaper models
- Push everything deterministic out of the model



This is not a Sarvam-specific trick. It is the general law of agent design, and it is where most of the cost and most of the errors go away at once.

MCP — connect your tools without exposing them

ONE-TIME CONFIG — CLAUDE DESKTOP, CLAUDE CODE, CURSOR, WINDSURF, ZED

```
{
  "mcpServers": {
    "sarvam": {
      "command": "uvx", "args": ["sarvam-mcp"],
      "env": { "SARVAM_API_KEY": "<YOUR_KEY>" }
    }
  }
}
```

Tool	What it does	Default model
sarvam_stt_transcribe	Audio file → transcript, five modes	saaras:v3
sarvam_tts_speak / _stream	Text → audio file or stream	bulbul:v3
sarvam_translate	Cross-language translation	mayura:v1
sarvam_transliterate	Script conversion	—
sarvam_identify_language	Language + script detection	—
sarvam_llm_complete	Chat completions	sarvam-105b



Two namespaces: sarvam_tools_* to actually call the APIs, sarvam_code_* for docs, endpoint shapes and starter projects while you build.



Evals

The module every workshop skips and every deployment needs

The buyer's second question

1

"Does it work?"

You demo it. It works. Everyone is pleased. This part is easy and it is not what closes the deal.

2

"How do you know it still works next month?"

This is the question that separates a pilot from a contract — and a documented eval harness is the only real answer to it.



Build the golden set from real data

30 cases from actual transcripts, not invented ones. Ten is enough to start. Real data surfaces failures your imagination will not.



Score what matters

Task success · tool-call accuracy · language correctness · hallucination rate · latency p50 and p95 · cost per task.



Run it on every prompt change

A prompt edit is a code change. It needs the same regression discipline. seed= makes results repeatable.



Guardrails belong here too

PII redaction before logging · refusal boundaries · human-in-the-loop escalation thresholds · full recording for compliance.

A minimal eval harness

PYTHON — run this on every prompt change

```
CASES = [  
    {"audio": "hi_emi_query.wav", "expect_tool": "get_emi_schedule",  
     "expect_lang": "hi-IN", "must_contain": ["EMI"]},  
    # ... 9 more from REAL transcripts  
]  
  
def run_evals():  
    results = []  
    for c in CASES:  
        out = agent(c["audio"], seed=42)          # seed = repeatable  
        results.append({  
            "tool_ok": out.tool_called == c["expect_tool"],  
            "lang_ok": out.language == c["expect_lang"],  
            "content_ok": all(k in out.text for k in c["must_contain"]),  
            "latency_ms": out.latency,  
            "cost_inr": out.cost,  
        })  
    return summarise(results)  # print pass rate, p95 latency, mean ₹/task
```



Now break one prompt deliberately and watch the harness catch it. That thirty seconds is what convinces a room that evals are not bureaucracy.

The wedding cook who lost the biryani at step 39

Forty steps. Soak, marinate, fry, layer, seal, dum.

At step 39 the gas ran out.

The question is not whether the gas will run out. It will. The question is whether you start again from the rice, or relight the stove and continue.

A cook who has to restart from step 1 every time something goes wrong cannot cater a wedding. He can only cook for four people.

40

Checkpointing is the difference between an agent that demos well for four steps and one that runs a fifty-step compliance review across an enterprise. Design for the gas running out.

Where you would actually deploy an agent

1

Compliance & risk review

Insurance. Review a policy document against IRDAI guidelines, flag deviations, cite the clause.

2

Customer onboarding

Fintech. KYC document extraction, validation, eligibility check, routing or escalation.

3

Loan application processing

BFSI. Multi-document triage, field validation, credit-policy check, decision or human escalation.

4

Contract review & redlining

Legal. Clause extraction, deviation from standard terms, suggested redlines with rationale.

5

Scheme eligibility & enrolment

Government. Citizen documents in 22 languages, eligibility rules, enrolment, status callback by voice.

6

Patient record processing

Healthcare. Handwritten and typed records, structured extraction, residency-compliant deployment.

7

Candidate screening

HR. Resume parsing at volume, structured scoring against a rubric, shortlist with reasoning.

8

Supply chain monitoring

Manufacturing. Document and message ingestion, exception detection, alerting with context.



Each of these is a vertical product, not a platform feature. That gap — between what a platform sells and what a specific buyer needs — is where a small team makes money.

Build it yourself, or deploy on a platform?

B Build on the raw APIs when

- The workflow logic is your actual differentiator
- You need margin at high volume
- You need a deployment shape the platform does not offer
- You are selling the software, not using it
- You have the engineering capacity to own reliability
- The vertical rules are too specific to configure

P Use Indus / Samvaad / Arya when

- Time to market matters more than unit cost
- You do not want to own state, retries and observability
- Volume does not justify the engineering
- You are the end user, not the vendor
- The buyer wants a supported platform, not your code
- You need enterprise compliance posture on day one

★ Neither answer is always right. But you cannot choose without the numbers — which is exactly what the next segment gives you.

“

Every agent decision, traceable.

Every step, recoverable.

That is not polish — it is the price of entry.

— End of Segment 05. Next: Kivi, Sarvam Code, and the tooling.